

Survey on Large Language Model-Enhanced Reinforcement Learning: Concept, Taxonomy, and Methods

Presenter: Hongming Zhang

Date: 2024.05.22

LLM-enhanced RL

- Definition

LLM-enhanced RL refers to the methods that utilize the multi-modal information processing, generating, reasoning, etc. capabilities of **pre-trained**, knowledge-inherent **AI models to assist the RL** paradigm.

- Two components

LLM (**model**): extensive pre-trained knowledge, in-context learning, reasoning ability, ...

RL (paradigm/**algorithm**): agent-environment interaction paradigm, policy learning, ...

LLM-enhanced RL

- Definition

LLM-enhanced RL refers to the methods that utilize the multi-modal information processing, generating, reasoning, etc. capabilities of **pre-trained**, knowledge-inherent **AI models to assist the RL** paradigm.

Benefits

- | | |
|-----|--|
| LLM | <ul style="list-style-type: none">• Powerful general knowledge |
| RL | <ul style="list-style-type: none">• Learning by itself through interaction, like humans |

Drawbacks

- Heavily **data-dependent** (supervised/unsupervised learning/finetuning, ...)
- **No self-improvement**, especially given a specific task
- **Sample inefficient** (exploration, estimation error, reward function, ...)
- **No understanding**/thinking, more like explore and cram up

RL4LLM or LLM4RL?

RL4LLM: **RLHF** (the series of ChatGPT), prompt optimization, ...

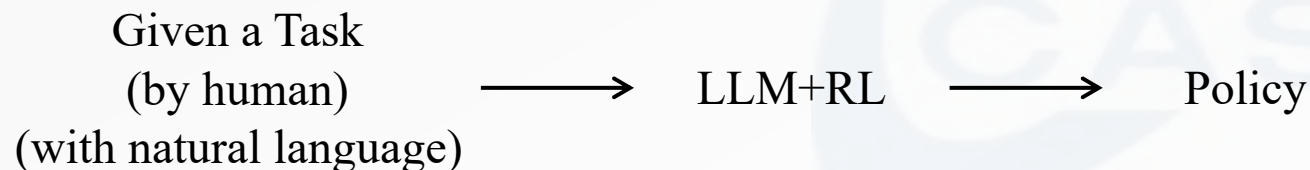
Why not RL4LLM?

- **Hard to train**/finetune (too many parameters, policy collapse, ...)
- Inference is **slow**, one task may not need a big net

What can LLM4RL assist in RL?

- Provide common knowledge and improve RL's **sample efficiency** (Reward function design, Generalization, Natural language understanding, Multi-modal, Long-Horizon Handling)

Grand prospect:



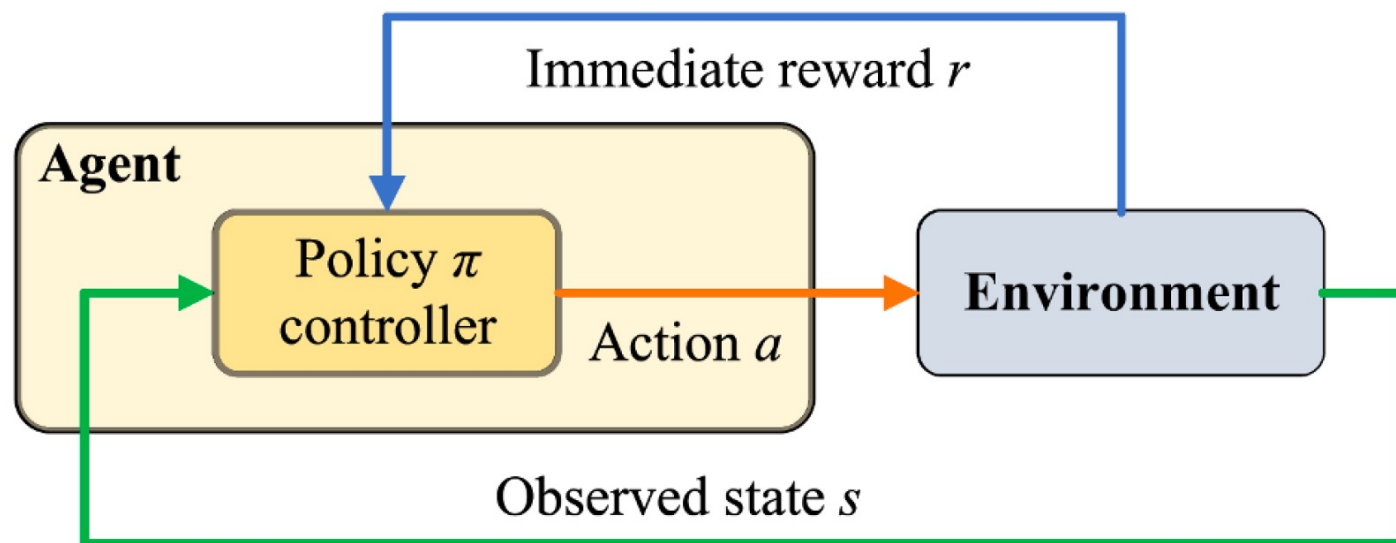


Fig. 1. Classical reinforcement learning paradigm.

LLM4RL:

- Information processor
- Reward designer
- Decision-maker
- Generator

LLM-enhanced RL

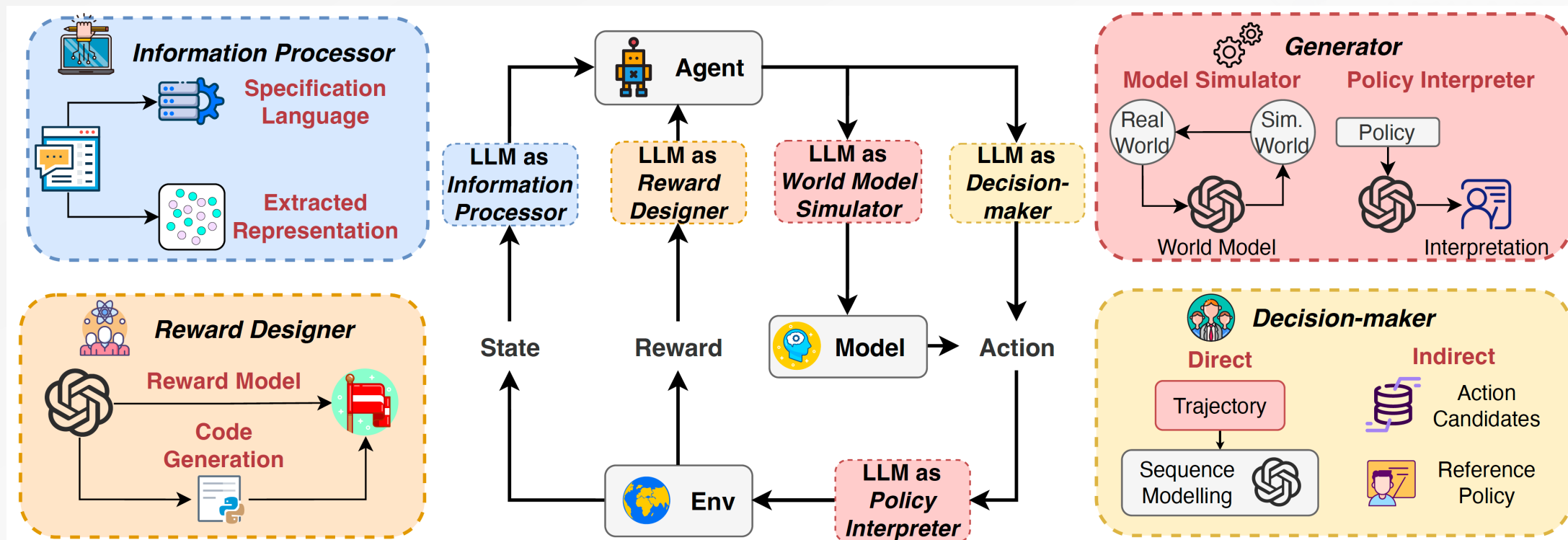


Fig. 2. Framework of LLM-enhanced RL in classical Agent-Environment interactions, where LLM plays different roles in enhancing RL.



- Information Processor
- Reward Designer
- Decision-Maker
- Generator



LLM as Information Processor

Feature Representation Extractor

- Frozen pre-trained model
- Further fine-tune the pre-trained model

Language Translator

- Instruction information translation
- Environment information translation

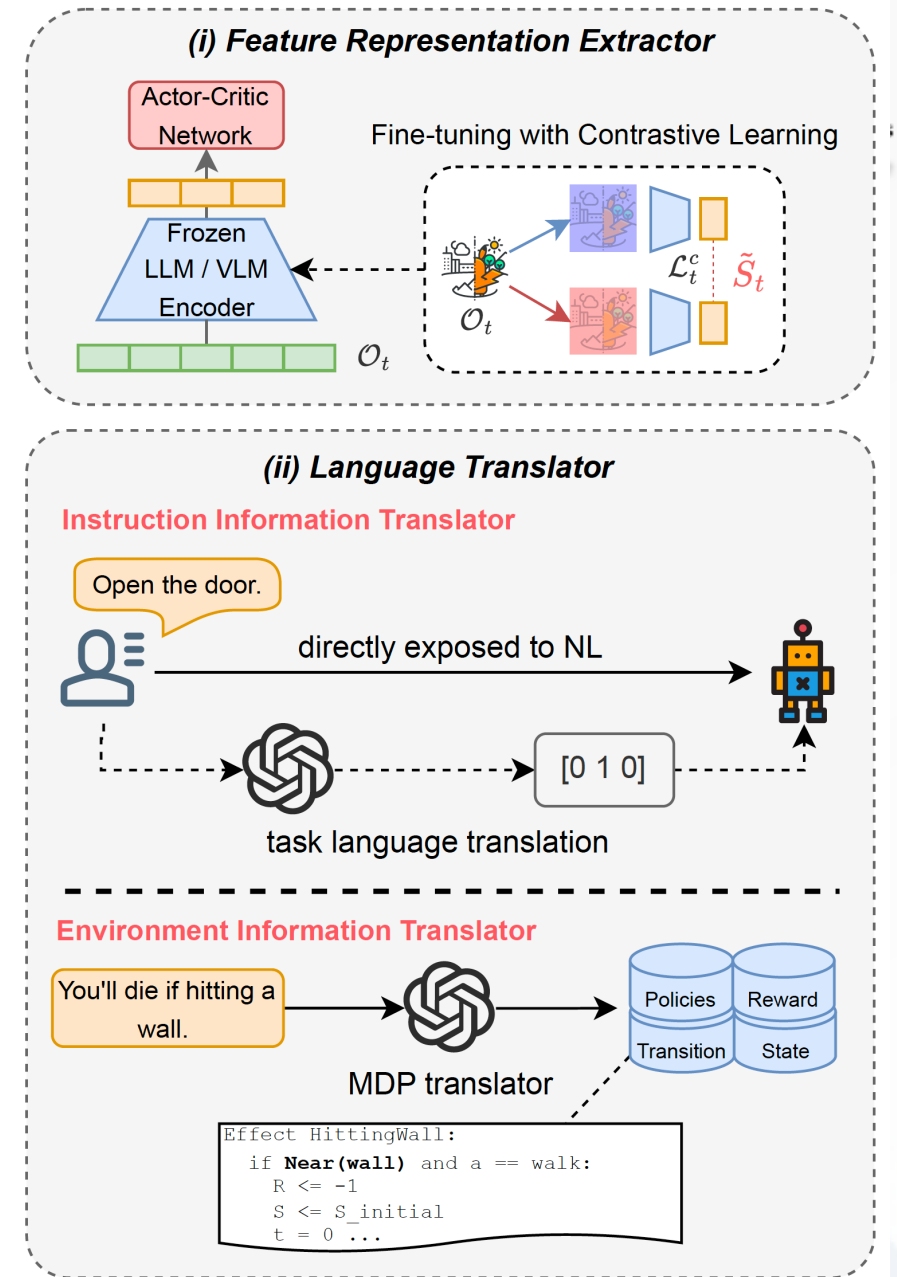
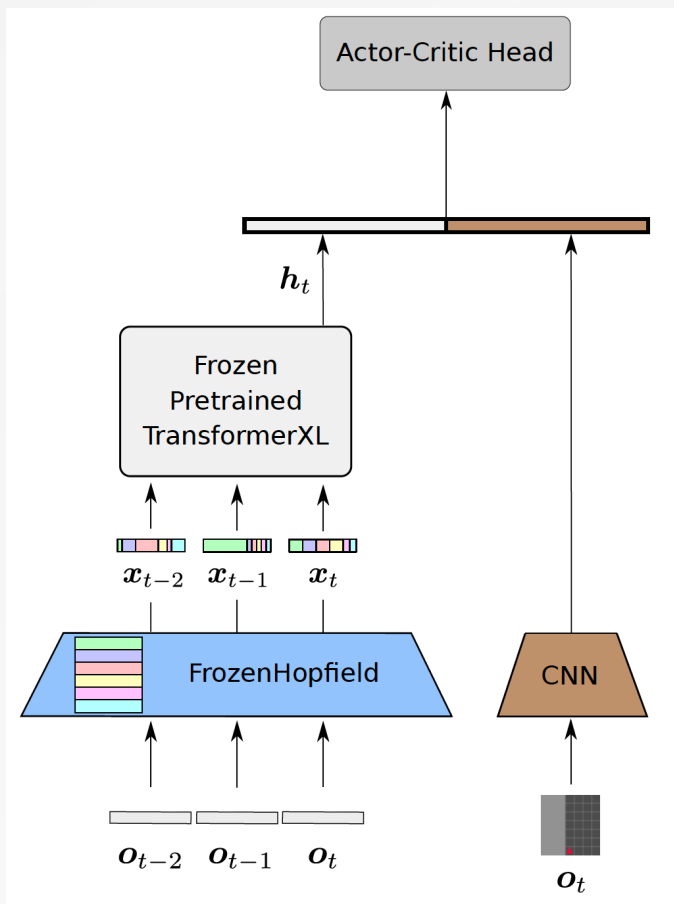


Fig. 3. LLM as an information processor.

History Compression via Language Models



Architecture of HELM (History Compression via Language Models)

- (i) the **memory mechanism** which utilizes a **pretrained language model**
- (ii) an encoder of the **current** timestep
- (iii) the **actor-critic** networks trained for policy optimization

$$\mathbf{x}_t^\top = \sigma \left(\beta \quad \mathbf{o}_t^\top \quad \mathbf{P}^\top \quad \mathbf{E}^\top \right) \mathbf{E}$$

$$\begin{bmatrix} \text{color bar} \end{bmatrix} = \sigma \left(\begin{bmatrix} \text{yellow square} & \text{grey rectangle} & \text{grey square} & \text{color bar} \end{bmatrix} \right) \begin{bmatrix} \text{color bar} \end{bmatrix}$$

The FrozenHopfield mechanism.

Contrastive Learning

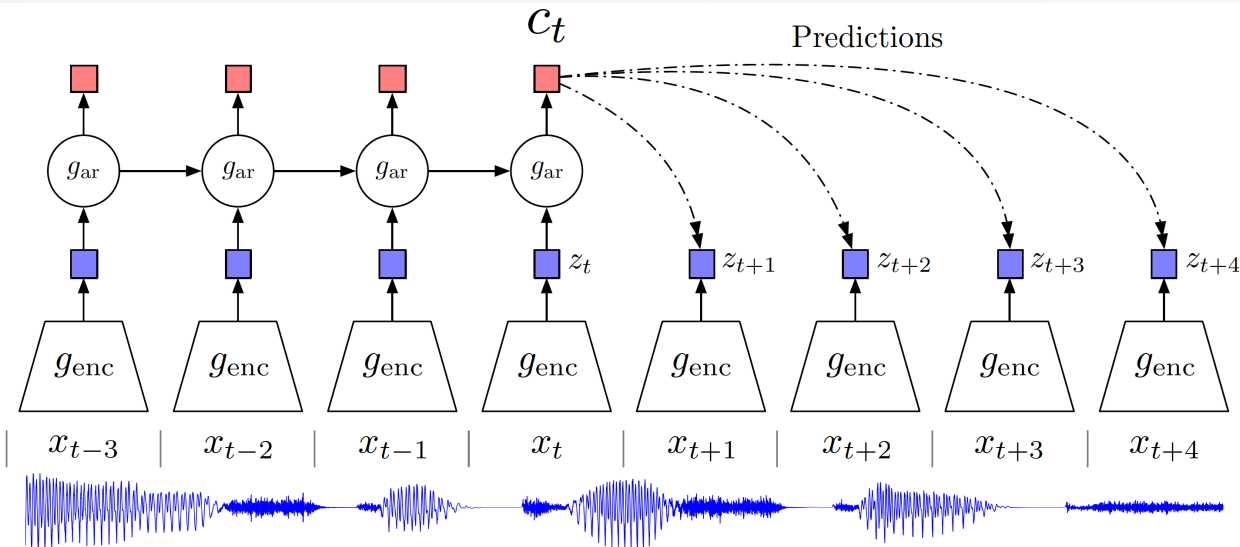


Figure 1: Overview of Contrastive Predictive Coding, the proposed representation learning approach. Although this figure shows audio as input, we use the same setup for images, text and reinforcement learning.

InfoNCE loss

$$\mathcal{L}_q = -\log \frac{\exp(q^T W k_+)}{\exp(q^T W k_+) + \sum_k^{K-1} \exp(q^T W k_i)} \quad (1)$$

one positive sample and K-1 negative samples

https://mp.weixin.qq.com/s?__biz=MzI4MDYzNzg4Mw==&mid=2247533903&idx=3&sn=ad7eecbfe48789bafd1aba1ae35cfc8c&chksm=ebb77d9bdcc0f48d208e037c5d4db2a2c929641adcf6eb1adaa9ae459f7d531c2af657a91593&scene=27

Augmented Temporal Contrast (ATC)

Decoupling of representation learning from reinforcement learning that performs as well as or better than end-to-end RL

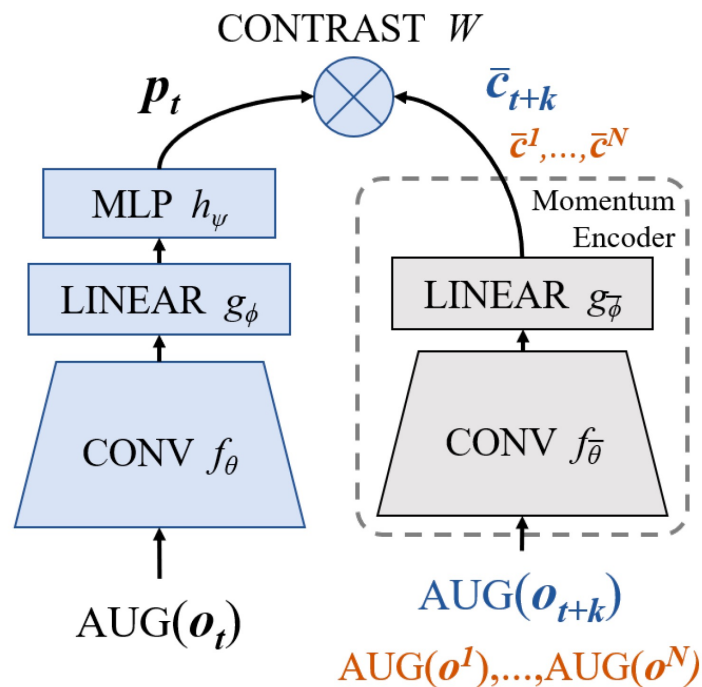


Figure 1. Augmented Temporal Contrast—augmented observations are processed through a learned encoder f_θ , compressor, g_ϕ and residual predictor h_ψ , and are associated through a contrastive loss with a positive example from k time steps later, processed through a momentum encoder.

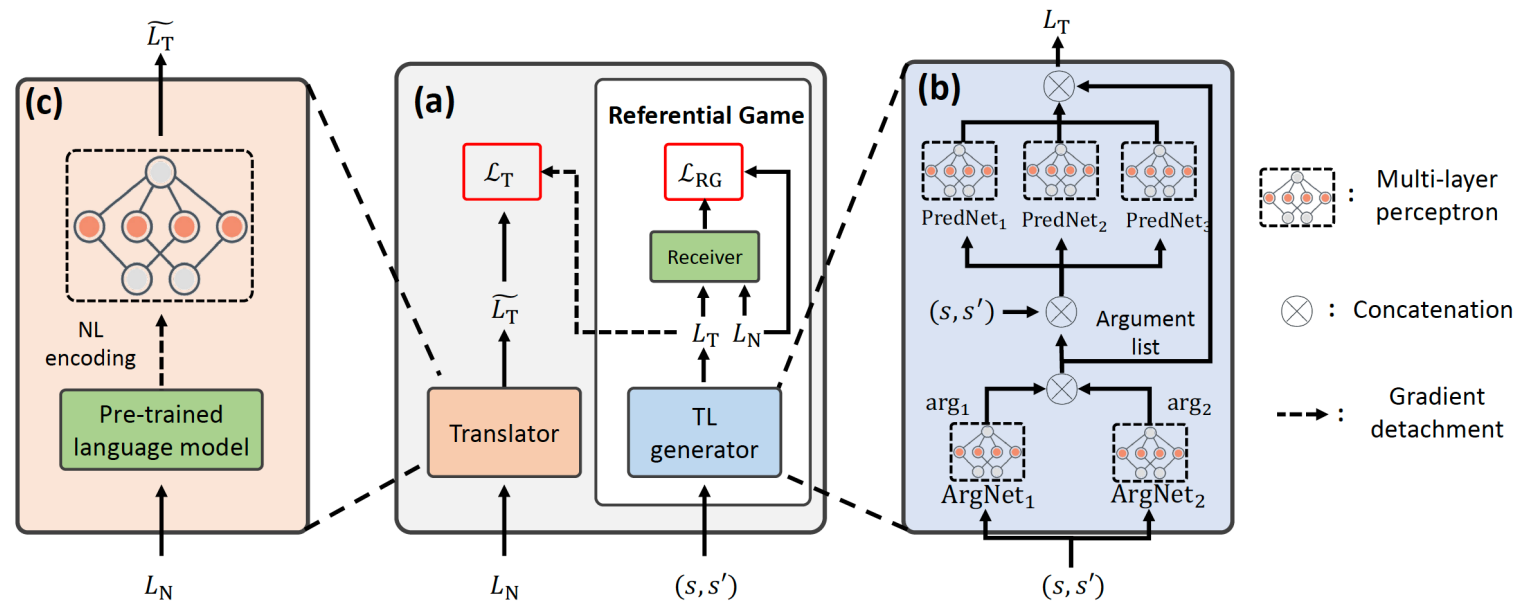
- update the encoder weights using only unsupervised learning (UL)
- train a control policy independently (SAC)

InfoNCE loss

$$\mathcal{L}^{ATC} = -\mathbb{E}_{\mathcal{O}} \left[\log \frac{\exp l_{i,i+}}{\sum_{o_j \in \mathcal{O}} \exp l_{i,j+}} \right]. \quad (2)$$

TALAR

Task Language with predicAte Representation:
translate human instructions from natural language (NL) into task language (TL)



Task dataset $D = \{(s, s', L_N)_i\}$

Each triplet contains
a **natural language instruction** L_N
and a **task state pair** (s, s') , where
 L_N describes state change from s to
 s' in natural language, e.g., move the
red ball to the blue ball.

Train Instruction-Following
Policy (IFP) with PPO.
During IFP training, the
translator's parameters are fixed.

$$\mathcal{L}_T(\phi) = \mathbb{E}_{(s, s', L_N) \sim \mathcal{D}} \left[-\log p_\phi(g_\theta(s, s') | b(T(L_N))) \right]. \quad (3)$$



- Information Processor
- Reward Designer
- Decision-Maker
- Generator



LLM as Reward Designer

Implicit Reward Model

- Direct Prompting
- Alignment Scoring

Explicit Reward Model

- Generate executable codes

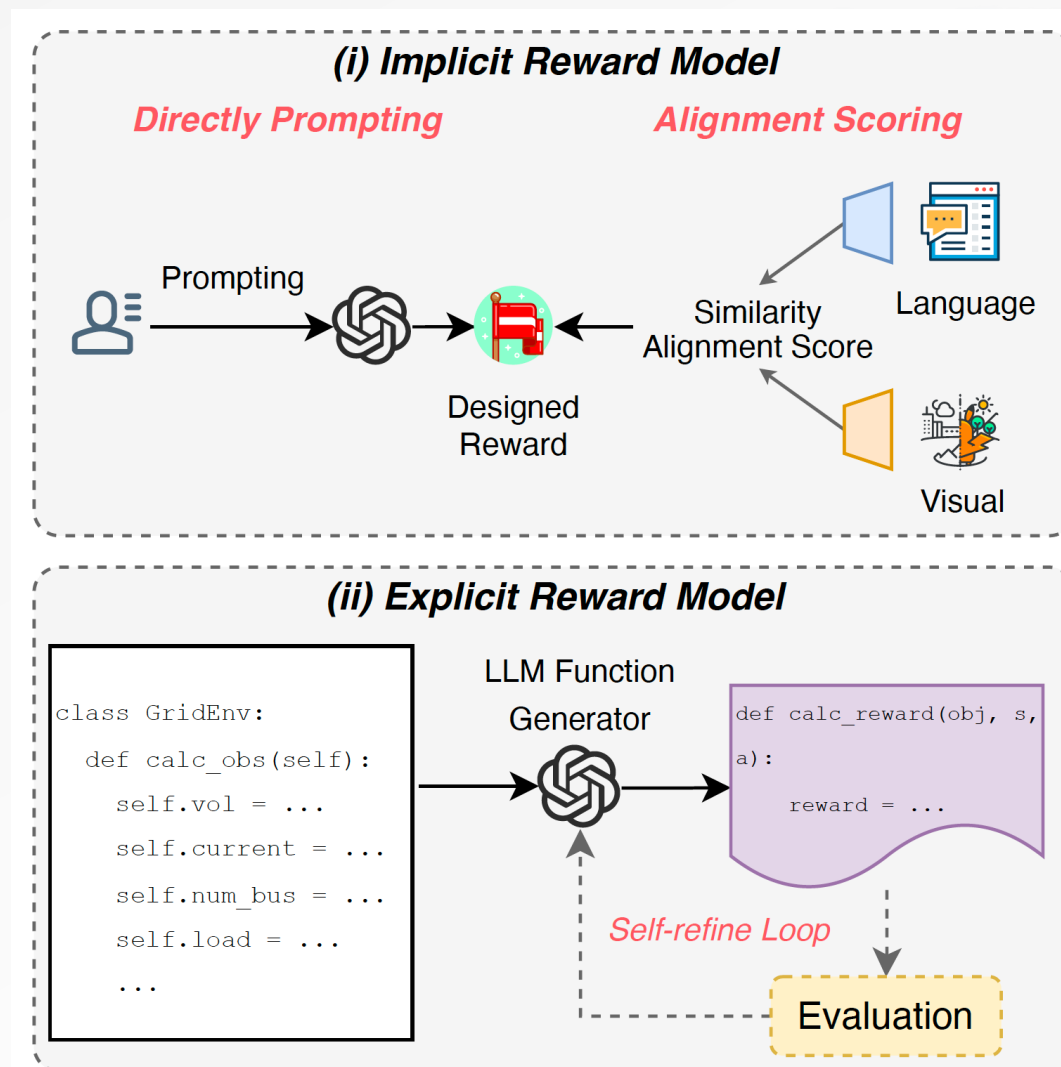
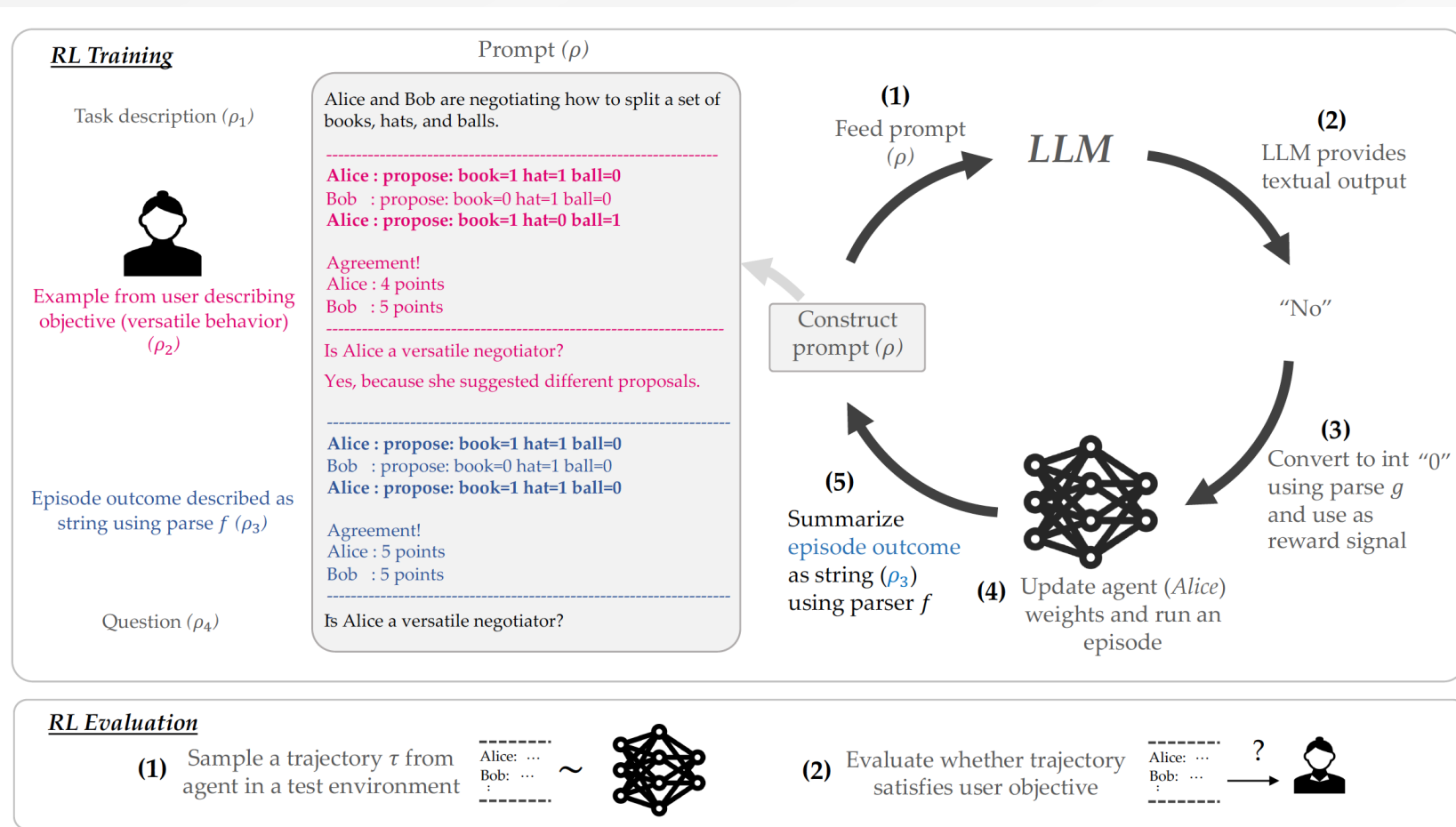


Fig. 4. LLM as a reward designer.

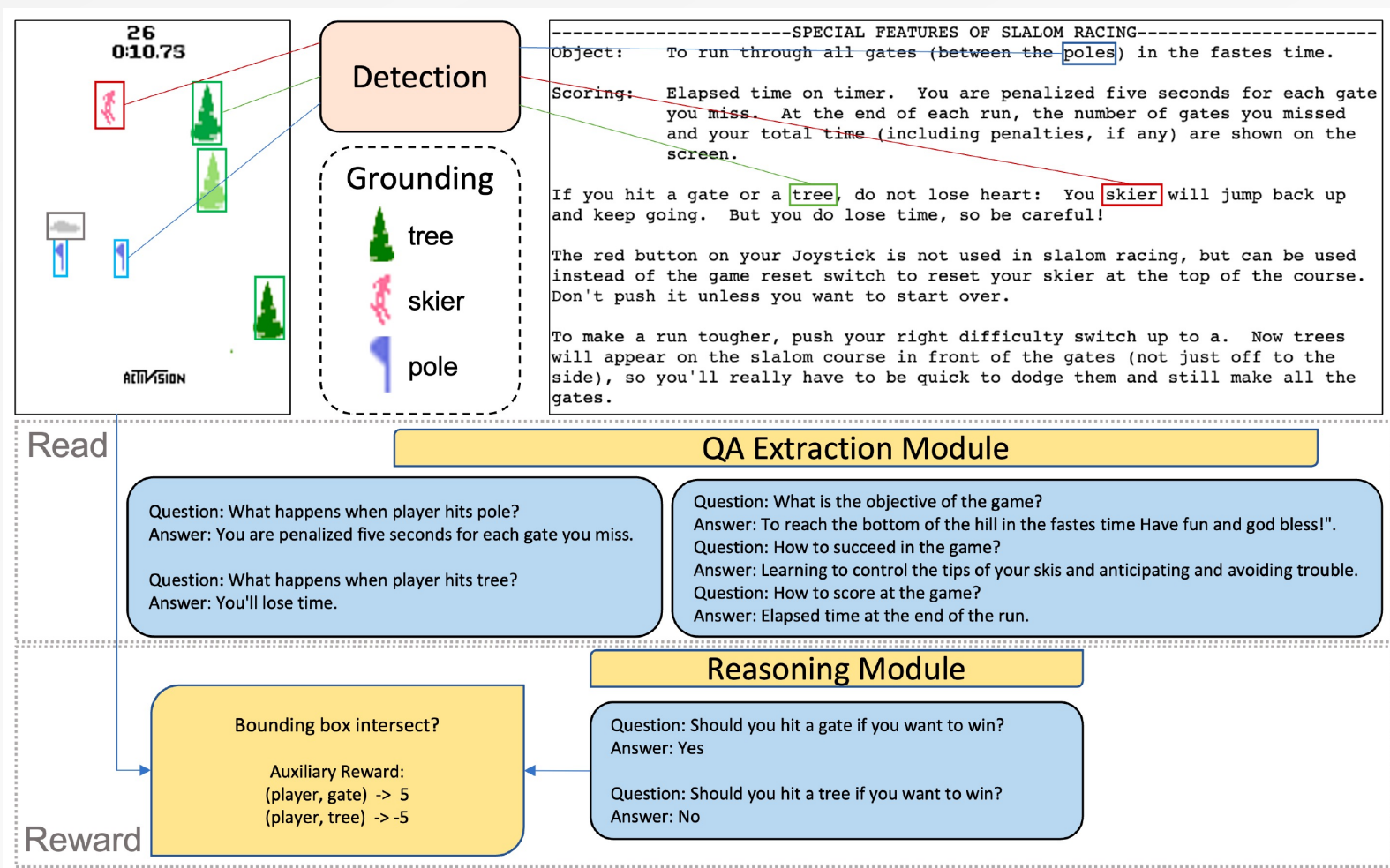
Reward Design with Language Models



- (1) Feed the **prompt** to the LLM
- (2) Take its **output**
- (3) **Parse** it into an integer using function g ; (handcrafted, task-specific parser. integer as the reward signal.)
- (4) The **RL** agent then updates its weights and rolls out an episode.
- (5) We parse the episode outcome into a string using f and **continue** training

Read and Reap the Rewards

Speeds up RL algorithms on Atari games by reading manuals released by the Atari game developers



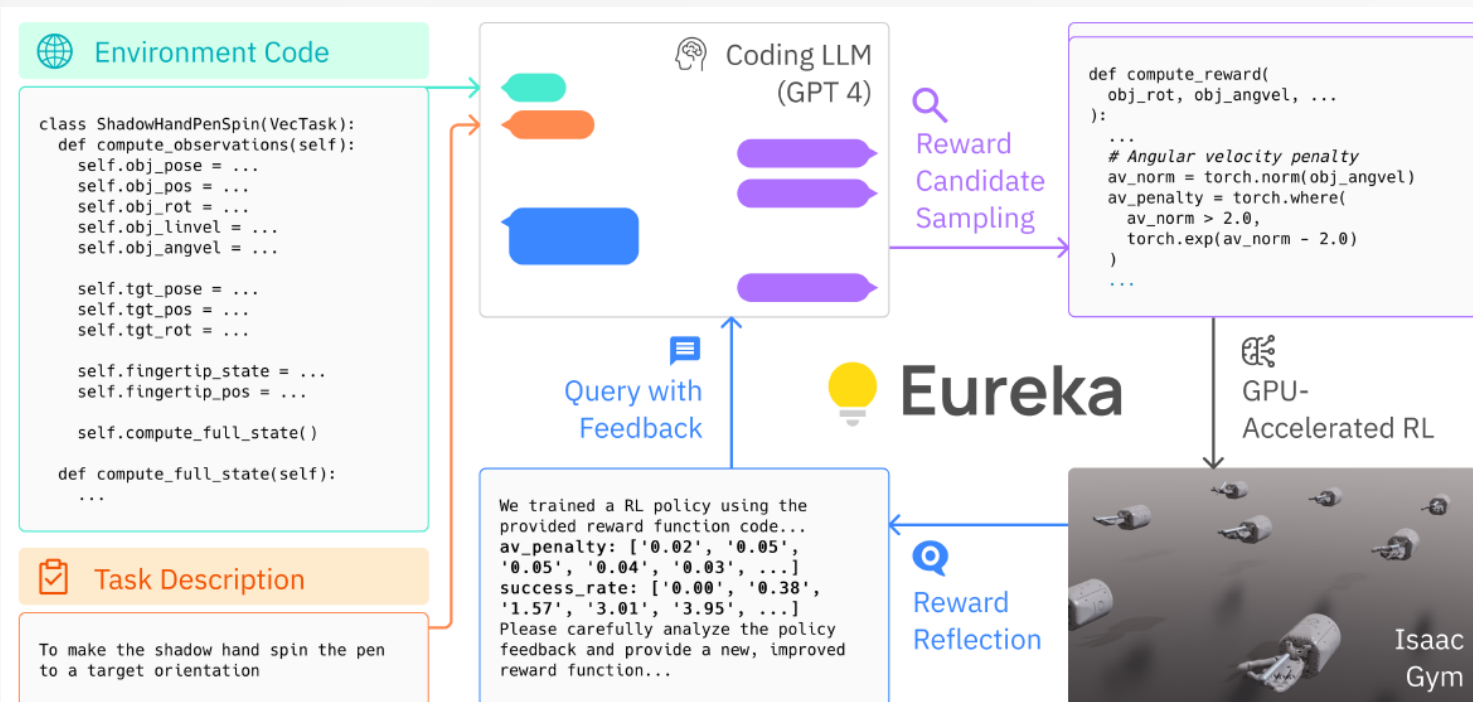
Receives the **current frame** in the environment, and the **instruction manual** as input.

After **object detection** and grounding, the **QA Extraction Module** extracts and summarizes relevant information

The **Reasoning Module** assigns auxiliary rewards to detected in-game events by reasoning with outputs from the QA Extraction Module.

The “Yes/No” answers are then mapped to +5/ – 5 auxiliary **rewards**.

Human-level **reward design** algorithm powered by LLMs.

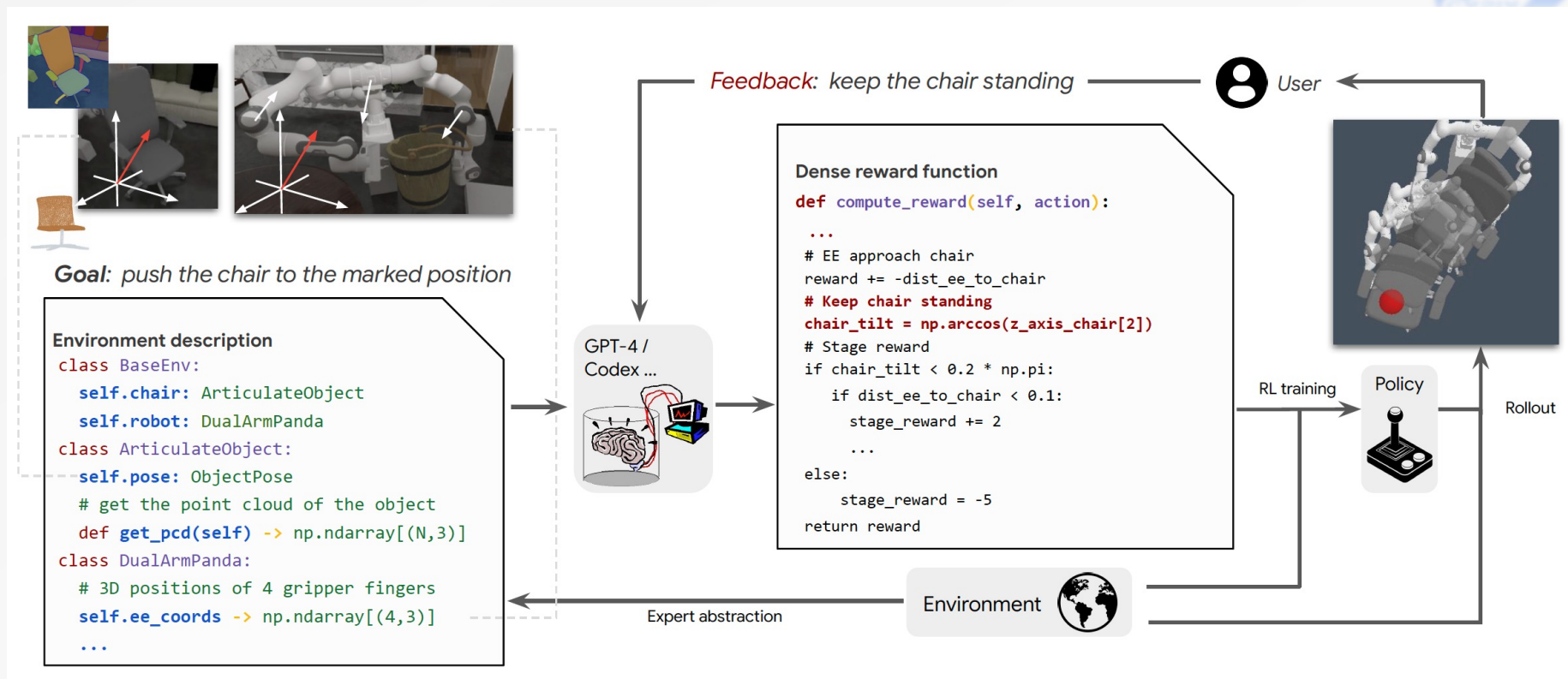


Specifying the mutation operator as a text prompt that suggests a few general ways to modify an existing reward code based on a textual summary of policy training

- (1) changing the **hyperparameter** of existing reward components,
- (2) changing the **functional form** of existing reward components, and
- (3) introducing **new reward components**

Figure 2: EUREKA takes unmodified environment source code and language task description as context to zero-shot generate executable reward functions from a coding LLM. Then, it iterates between reward sampling, GPU-accelerated reward evaluation, and reward reflection to progressively improve its reward outputs.

Text2Reward



Expert Abstraction provides an abstraction of the environment as a hierarchy of Pythonic classes.

User Instruction describes the goal to be achieved in natural language.

User Feedback summarizes the failure mode or their preferences, which are used to improve the **reward code**.

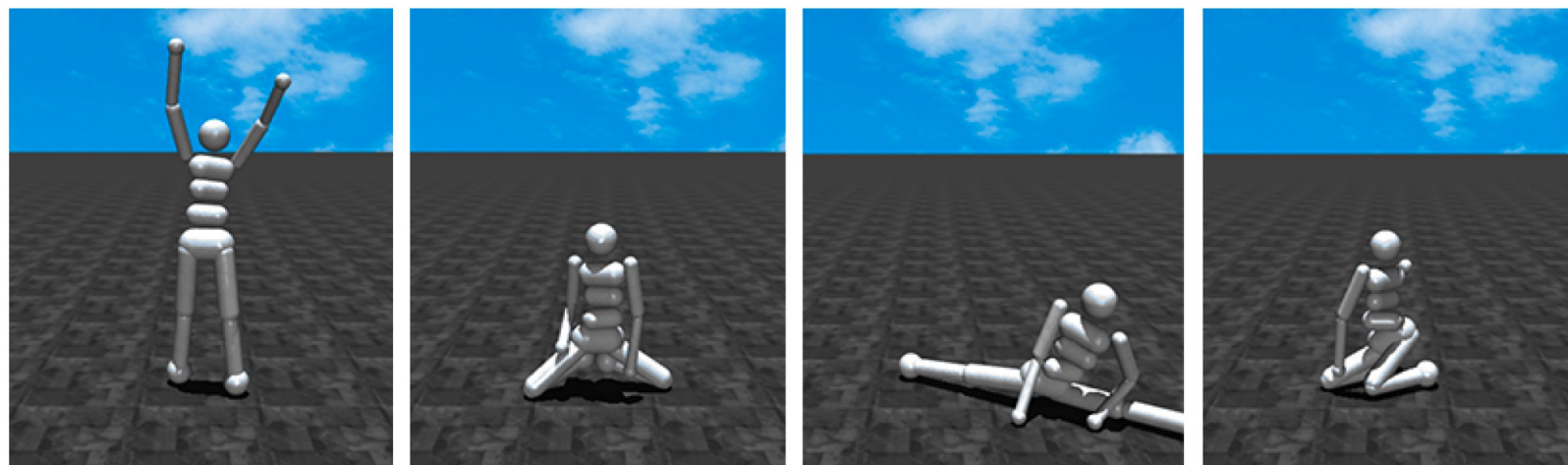
Using pretrained vision-language models (VLMs) as zero-shot reward models (RMs) to specify tasks via natural language

CLIP as a reward model

- the CLIP image encoder as a state encoder
- the CLIP language encoder as a task encoder

$$R_{\text{CLIP}}(s) = \frac{\text{CLIP}_L(l) \cdot \text{CLIP}_I(\psi(s))}{\|\text{CLIP}_L(l)\| \cdot \|\text{CLIP}_I(\psi(s))\|}. \quad (2)$$

RL: DQN or SAC



(1) stand with raised arms, (2) sit in a lotus position, (3) do the splits (4) kneel on the ground



- Information Processor
- Reward Designer
- Decision-Maker
- Generator



LLM as Decision-Maker

Direct Decision-Maker

Indirect Decision-Maker

- Action Candidates
- Reference Policy

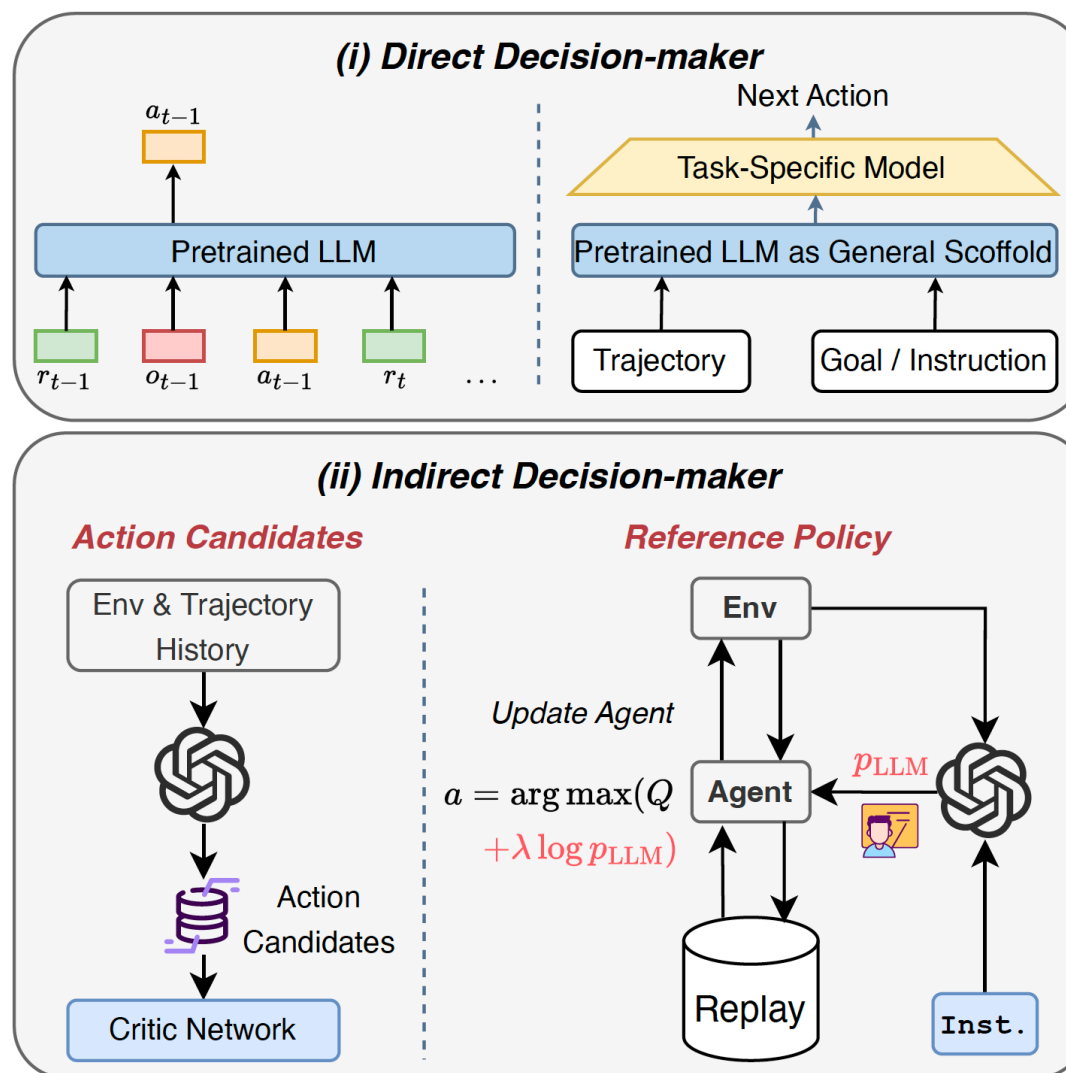
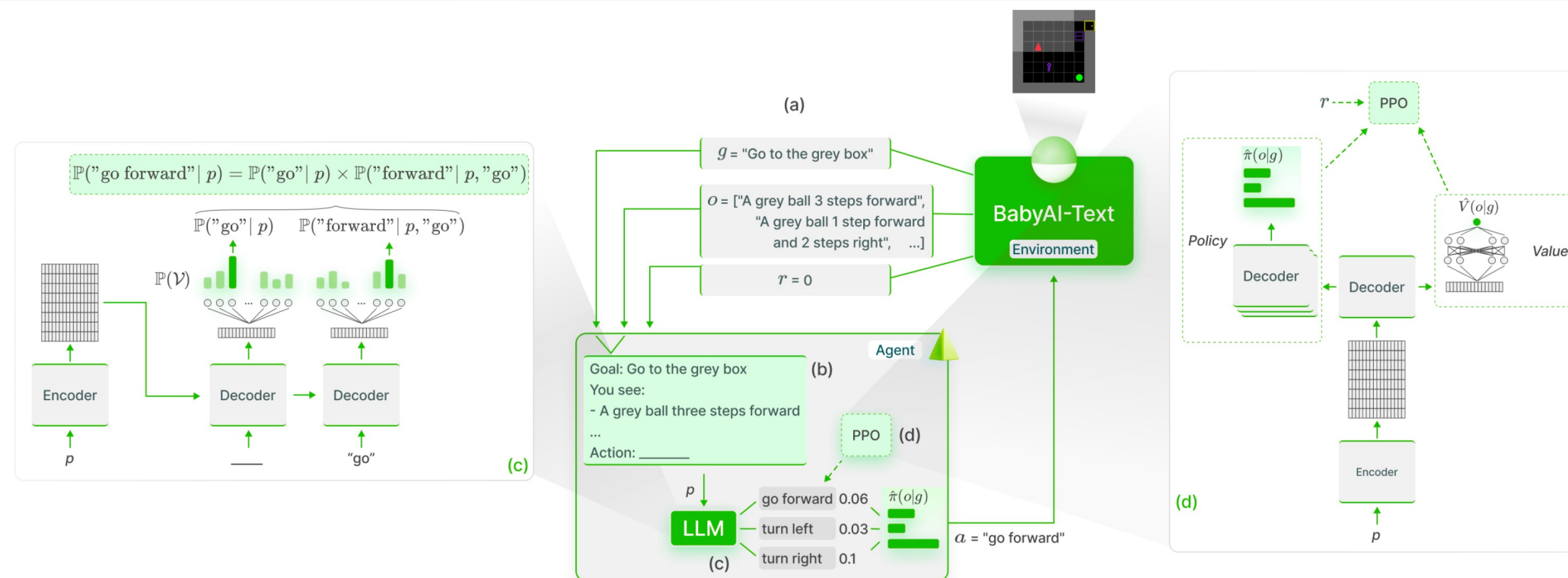


Fig. 5. LLM as a decision-maker.



LLM as agent policy, trained using PPO on BabyAI-Text

Flan-T5 780M

(a) goal description, description of the agent observation, reward

(b) sent to LLM

(c) generate a representation of the prompt, compute the conditional probability of tokens composing the action

(d) finetune LLM using PPO

Language Models for Motion Control (LaMo)

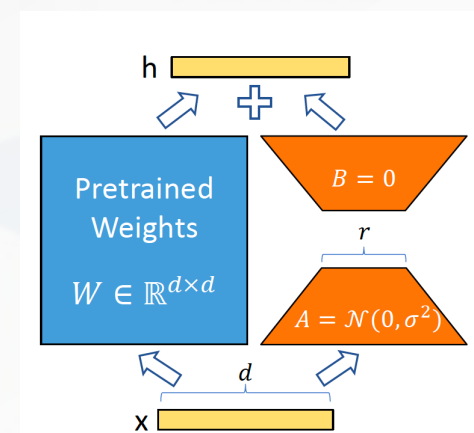
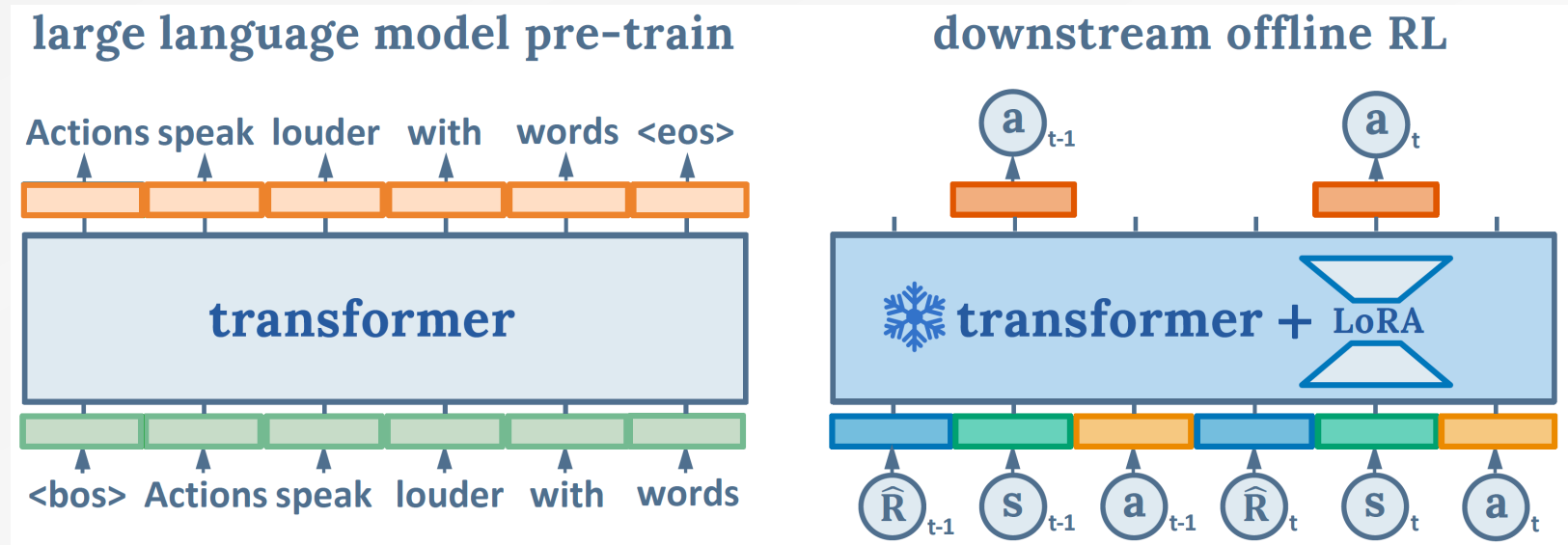


Figure 1: Our reparametrization. We only train A and B .

- 1) **pre-training** LMs on language tasks
- 2) freezing the pre-trained attention layers, replacing linear projections with **MLPs**, and using **LoRA** to adapt to RL tasks, apply the **language loss** during the **offline RL** stage as a regularizer.

InstructRL

Enables humans to specify strategies they expect from their AI partners through natural language instructions

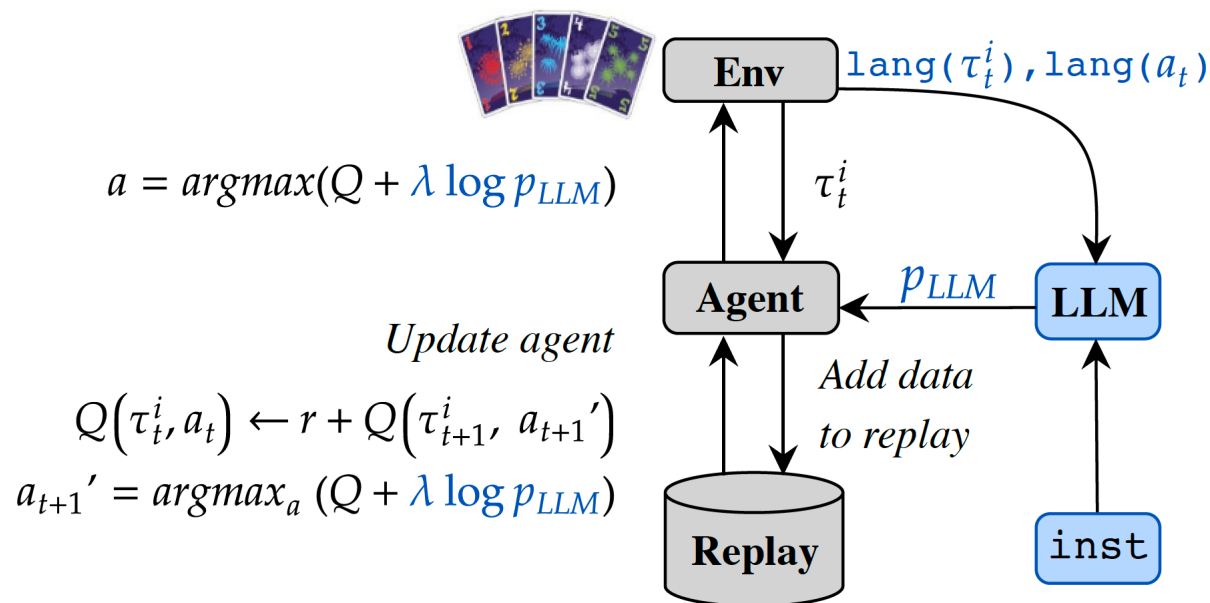


Figure 2. InstructQ. The differences between instructQ and normal Q-learning is highlighted in blue.

Use **pretrained LLM** to generate a prior policy conditioned on the human instruction and use the prior to **regularize** the RL objective

$$p_{LLM}[\operatorname{lang}(a_t) | \operatorname{lang}(\tau_t^i), inst]$$

DQN:

$$Q_{\theta}(\tau_t^i, a_t) \leftarrow r_t + \gamma Q_{\theta}(\tau_{t+1}^i, a'_{t+1}),$$

where $a'_{t+1} = \operatorname{argmax}_a [Q(\tau_{t+1}^i, a) + \lambda \log p_{LLM}(\tau_{t+1}^i, a)]$.

PPO:

$$\mathbb{E}_{\tau \sim \pi_{\theta}} \sum_t [-\log \pi_{\theta}(a_t | \tau_t^i) A_t + \lambda \operatorname{KL}[\pi_{\theta}(\tau_t^i) || p_{LLM}(\tau_t^i)]]$$

A framework that uses Pre-Trained Language Models for Interactive Decision-Making

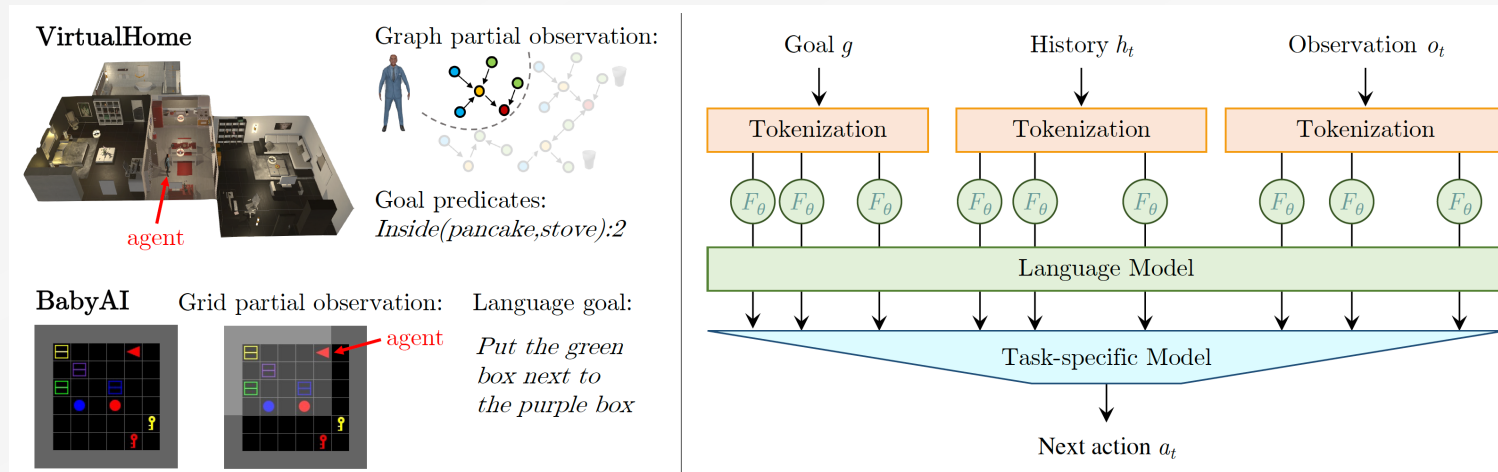


Figure 1: **Environments (left):** Different environments have different types of observations and goals. **Our approach (right):** We use pre-trained LMs as a general framework for interactive decision-making by converting policy inputs into sequential data. Such a method enables effective combinatorial generalization to novel tasks.

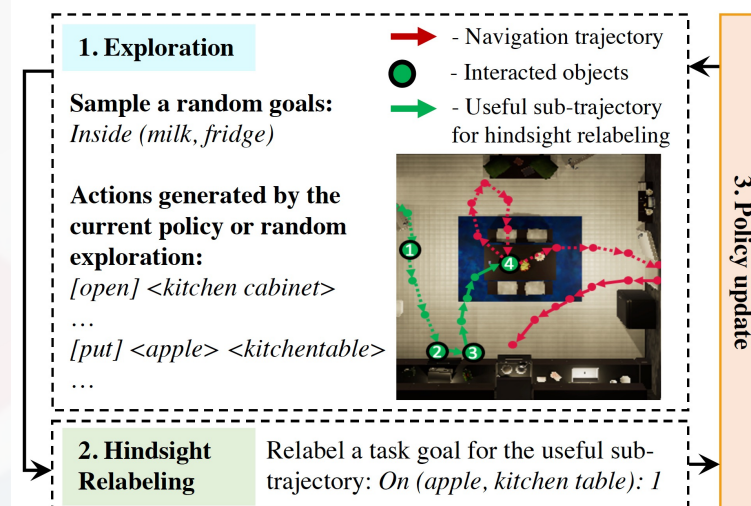


Figure 2: **LID with the active data gathering procedure.** By iteratively repeating the exploration, hindsight relabeling, and policy update, LID with active data gathering can learn an effective policy without using pre-collected expert data.

Initializing policies with LMs and fine-tuning them via **behavior cloning**

integrate an Active Data Gathering (**ADG**) procedure: agents iteratively interact with the environment, relabel past “failed” experiences with new goals, and update their policies

Gato

Works as a multi-modal, multi-task, multi-embodiment **generalist** policy.

Serialize all data into a flat sequence of tokens
Text tokens, discrete and continuous values, and actions can be directly set as targets after tokenization.

Gato was trained **offline** in a purely **supervised** manner.

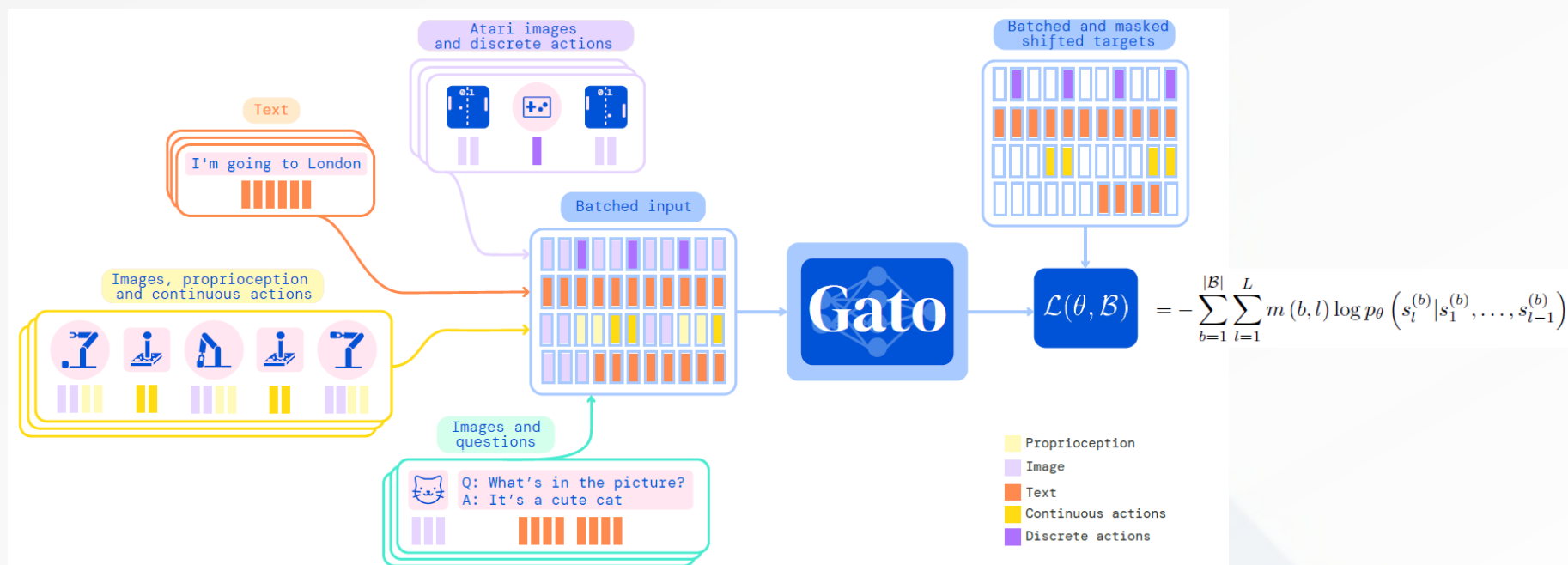


Figure 2: **Training phase of Gato.** Data from different tasks and modalities is serialized into a flat sequence of tokens, batched, and processed by a transformer neural network akin to a large language model. Masking is used such that the loss function is applied only to target outputs, i.e. text and various actions.



- Information Processor
- Reward Designer
- Decision-Maker
- Generator



LLM as Generator

World Model Simulator

- Trajectory Rollout
- Dynamics Representation Learner

Policy Interpreter

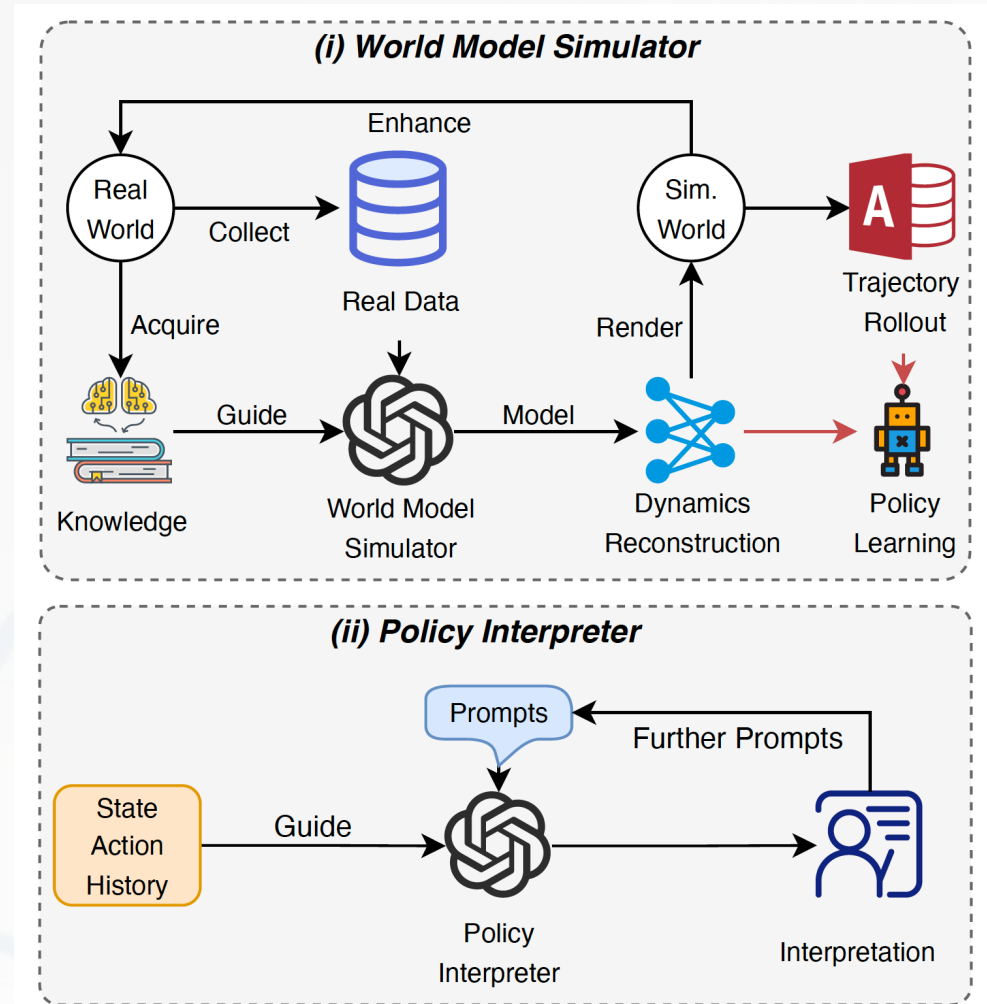


Fig. 6. LLM as a generator.

Decision Transformer

Casts the problem of RL as **conditional sequence modeling**

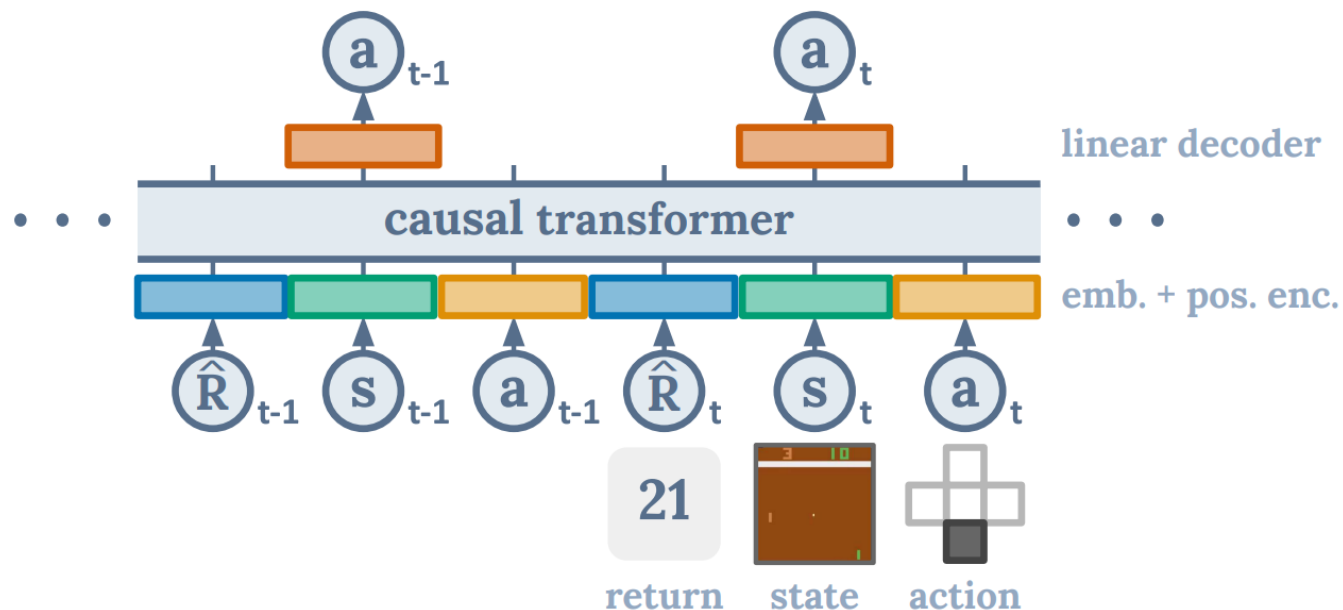
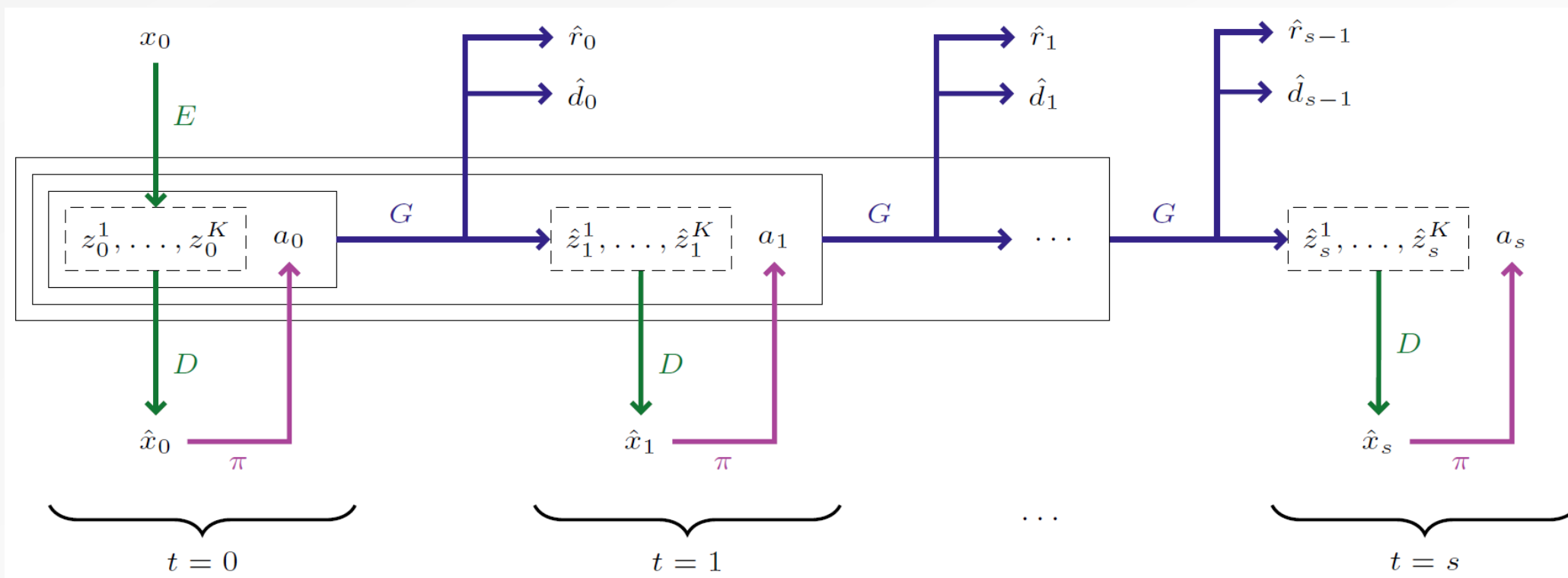


Figure 1: Decision Transformer architecture¹. States, actions, and returns are fed into modality-specific linear embeddings and a positional episodic timestep encoding is added. Tokens are fed into a GPT architecture which predicts actions autoregressively using a causal self-attention mask.

- Bypass the need for bootstrapping for long term credit assignment.
- Achieve policy improvement **without the need for dynamic programming**, matches or exceeds the performance of state-of-the-art model-free offline RL algorithms.

Imagination with auto-Regression over an Inner Speech (IRIS): VQ-VAE + Transformer + DreamerV2

Transformer is a more data-efficient world model compared with Dreamer



Masked World Models (MWM)

MAE+DreamerV2: A visual model-based RL algorithm that decouples visual representation learning and dynamics learning.

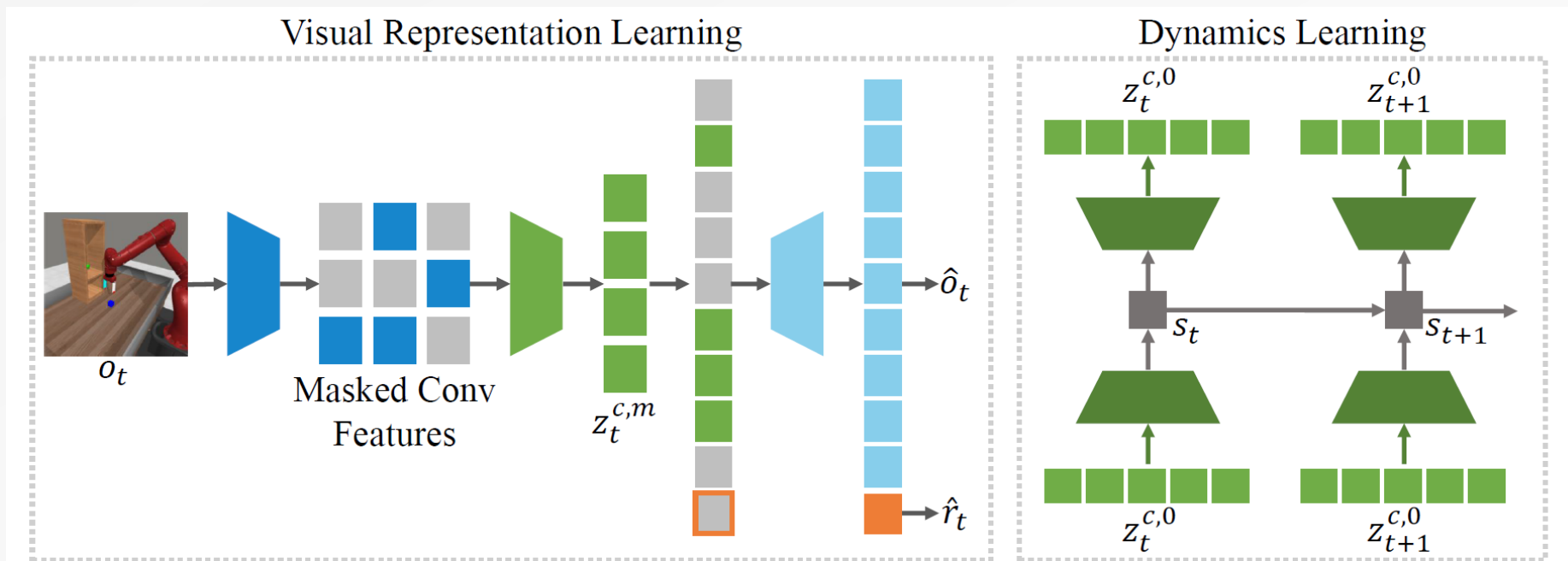


Figure 1: Illustration of our approach. We continually update visual representations and dynamics using online samples collected from environment interaction, by repeating iterative processes of training (Left) an autoencoder with convolutional feature masking and reward prediction and (Right) a latent dynamics model in the latent space of the autoencoder. We note that autoencoder parameters are not updated during dynamics learning.

Discussions

Applications

- Robotics
- Autonomous Driving
- Energy Management
- Healthcare Recommendation

Challenges

- Dependence on LLM's Capabilities
- Interaction Efficiency
- Ethical, Legal, and Safety Concerns

Opportunities

- RL: multi-agent RL, safe RL, transfer RL, explainable RL
- LLM: integration of LLMs with tools such as mathematical solver and internet browser, retrieval-augmented generation (RAG) techniques

Discussions

- Representation

Differences between Transformer and CNN/RNN

CNN: local, spatial invariance

RNN: sequential, time invariance

Transformer: global, sequential

- Model for POMDP

Transformer for representation + Transformer for dynamics learning

- Online learning using Transformer

Value function/Policy function

- LLM augmented RL

Reward function/Preference-based RL

Generate experience/data augmentation

Guide exploration

[Benchmark? tasks, algorithms ...](#)

Sumers, Theodore R., Mark K. Ho, Robert D. Hawkins, Karthik Narasimhan, and Thomas L. Griffiths. "Learning rewards from linguistic feedback." In Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, no. 7, pp. 6002-6010. 2021.

Liang, Jacky, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. "Code as policies: Language model programs for embodied control." In 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 9493-9500. IEEE, 2023.

Song, Chan Hee, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. "Llm-planner: Few-shot grounded planning for embodied agents with large language models." In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 2998-3009. 2023.

Kim, Changyeon, Younggyo Seo, Hao Liu, Lisa Lee, Jinwoo Shin, Honglak Lee, and Kimin Lee. "Guide Your Agent with Adaptive Multimodal Rewards." Advances in Neural Information Processing Systems 36 (2024).

Yu, Wenhao, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montserrat Gonzalez Arenas, Hao-Tien Lewis Chiang et al. "Language to Rewards for Robotic Skill Synthesis." In 7th Annual Conference on Robot Learning. 2023.

Madaan, Aman, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon et al. "Self-refine: Iterative refinement with self-feedback." Advances in Neural Information Processing Systems 36 (2023).

Zitkovich, Brianna, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu et al. "Rt-2: Vision-language-action models transfer web knowledge to robotic control." In Conference on Robot Learning, pp. 2165-2183. PMLR, 2023.

Brohan, Anthony, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz et al. "Do as i can, not as i say: Grounding language in robotic affordances." In Conference on robot learning, pp. 287-318. PMLR, 2023.

Seo, Younggyo, Kimin Lee, Stephen L. James, and Pieter Abbeel. "Reinforcement learning with action-free pre-training from videos." In International Conference on Machine Learning, pp. 19561-19579. PMLR, 2022.

Das, Devleena, Sonia Chernova, and Been Kim. "State2explanation: Concept-based explanations to benefit agent learning and user understanding." Advances in Neural Information Processing Systems 36 (2023): 67156-67182.



THANKS!

